Discount Strategy What-If Scenario Analysis Dashboard

Author: Pam Sani George (P.S.G)

CONTENTS

1 Project Introduction

2 Project Description

3 Project Overview

4 Python Data Processing

5 PostgreSQL Database Integration

6 SQL Data Validation

7 Power BI Dashboard

8 About the Analyst

1. Project Introduction

Overview

An Interactive Pricing Simulation for Revenue & Profit Analysis.

Project Title

Discount Strategy What-If Scenario Analysis Dashboard

Type

Data Analytics Portfolio Project

Analyst

Pam Sani George

Brand

Smart-P Analytics

Slogan

Built on Resilience. Powered by Data

Dataset

Sample Super Store — 9,994 Rows × 21 Columns

2. Project Description

What This Project Does

This project demonstrates a fully interactive Profit Simulation Dashboard built in Microsoft Power BI, designed to help businesses analyze how pricing and cost changes impact profitability.

The dashboard allows decision makers to simulate different business scenarios using dynamic parameters and instantly see the financial impact through automated calculations and visual insights.

This solution is designed for business executives, financial analysts, and strategy teams who need quick insights to support data-driven decision making.

Key Features

Interactive KPI Monitoring

Real-time tracking of key performance indicators across scenarios

Profit Simulation Analysis

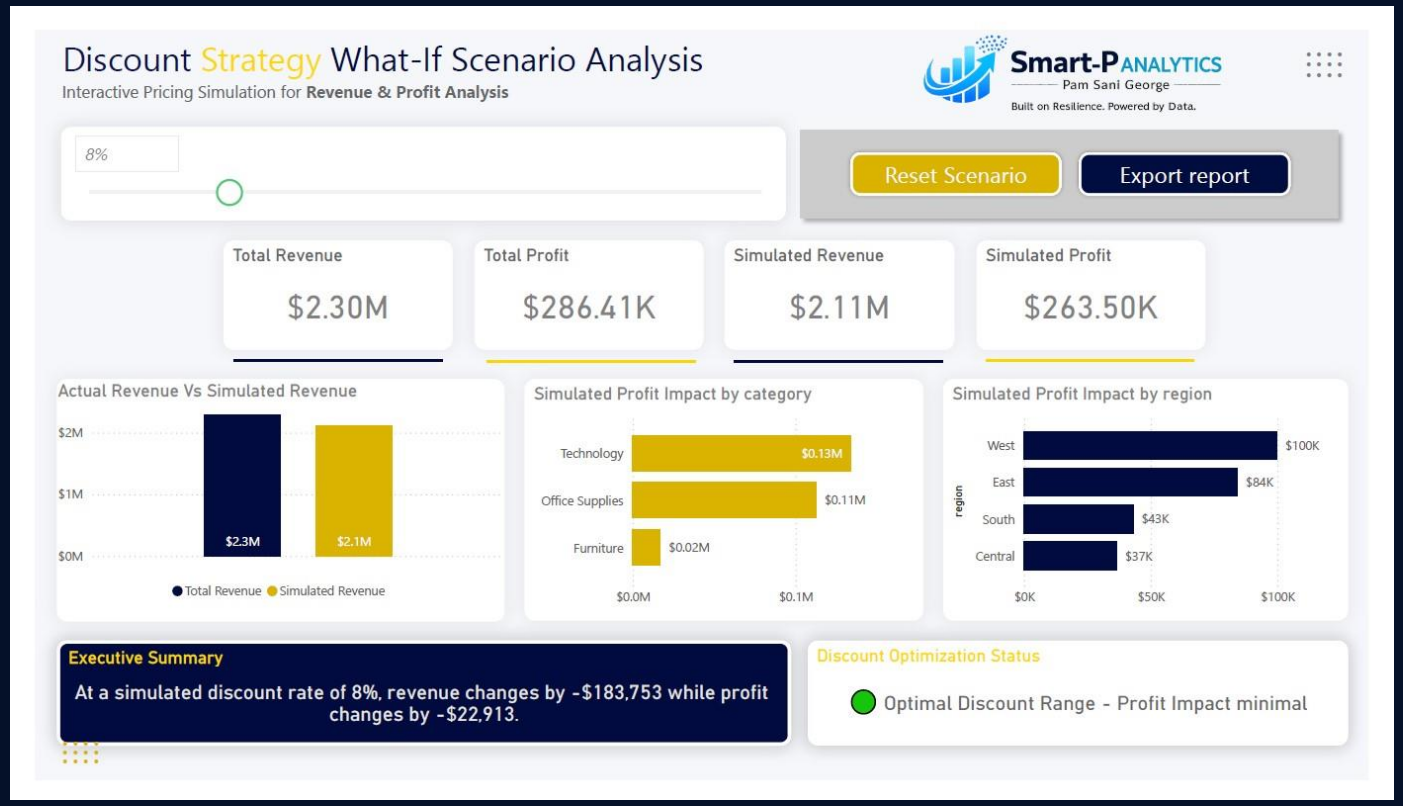
Dynamic simulation of pricing and cost changes on profitability

Automated Executive Summaries

Instant narrative insights generated from scenario results

Scenario Reset Functionality

Ability to reset and re-run multiple what-if scenarios



3. Project Overview

Business Problem

Businesses often struggle to understand how small changes in pricing or operational cost affect profitability.

This project provides an interactive analytics dashboard that allows users to simulate price and cost changes and immediately visualize how those changes impact overall profit.

Using advanced DAX calculations and interactive controls, the dashboard dynamically updates KPIs, charts, and executive insights to support faster and smarter decision making.

Core Capabilities Demonstrated

Business Intelligence Design

Professional dashboard layout following executive reporting standards

DAX Calculations

Advanced formula-driven dynamic measures and KPIs

Interactive Dashboard Development

What-if parameter slicers and real-time visual updates

Executive-Level Reporting

Clean, actionable insight delivery for senior stakeholders

End-to-End Data Analytics Pipeline

This project showcases a complete end-to-end data analytics solution built using Python, PostgreSQL, SQL, and Power BI.

The workflow begins with raw dataset processing in Python, where the data is cleaned, structured, and prepared for analysis. The cleaned dataset is then stored in a PostgreSQL database, queried using SQL, and connected to Power BI for visualization.

The Power BI dashboard includes a dynamic what-if scenario analysis that allows users to simulate discount strategies and instantly evaluate their impact on revenue and profitability.

Discount Strategy What-If Scenario Dashboard

Interactive Pricing Simulation for Revenue & Profit Analysis

1. Creating a Safe SQLAlchemy engine for PostgreSQL DataBase

```python
# Importing Operating System & SQLAlchemy Packages
import os
from sqlalchemy import create_engine, text
from sqlalchemy.engine import URL

# IMPORTANT: for security, I use environment variables
db_user = os.getenv("PGUSER", "postgres")
db_password = os.getenv("PGPASSWORD", "PamXXSxxGeorge")
db_host = os.getenv("PGHOST", "localhost")
db_port = os.getenv("PGPORT", "5432")
db_name = os.getenv("PGDB", "Scenario_DB")

connection_url = URL.create(
        "postgresql+psycopg2",
        username = db_user,
        password = db_password,
        host = db_host,
        port = db_port,
        database = db_name
)

engine = create_engine(connection_url)
print("Engine created sucessfully✅. Test connection below...")

# Quick Test
with engine.connect() as conn:
    r = conn.execute(text("SELECT 1")).scalar()
    print("Test query returned:", r)
```

```
Engine created sucessfully✅. Test connection below...
Test query returned: 1
```

4. Python Data Processing

Data Cleaning & Transformation

Python (Jupyter Notebook) was used for initial data ingestion, cleaning, and transformation to ensure the dataset was ready for database storage and analytics.

Fixing Data Types for Database Readiness

The following data type corrections were applied to ensure consistency and database compatibility:

Date Columns

order_date and ship_date converted to datetime64[ns] using pd.to_datetime()

Postal Code

Converted to string type for safe Database storage

Numeric Fields

Row_id retained as int64; all ID and name fields stored as object

4. Fixing Data Types for Database Readiness

```python
# Convert Date Colums
df['order_date'] = pd.to_datetime(df['order_date'], errors = 'coerce')
df['ship_date'] = pd.to_datetime(df['ship_date'], errors = 'coerce')

# Convert Postal_code to string (safe for database storage)
df['postal_code'] = df['postal_code'].astype(str)

print("Data type corrections completed ✓")
print("\nUpdated Data Types:")
print(df.dtypes)
```

```
Data type corrections completed ✓

Updated Data Types:
ï»¿row_id                int64
order_id                object
order_date       datetime64[ns]
ship_date        datetime64[ns]
ship_mode               object
customer_id             object
customer_name           object
segment                 object
ountry                  object
city                    object
state                   object
postal_code             object
region                  object
product_id              object
category                object
sub_category            object
product _name           object
sales                  float64
quantity                 int64
discount               float64
profit                 float64
dtype: object
```

5. PostgreSQL Database Integration

Creating the SQL Database Table with Python

Python was used to programmatically create and manage the PostgreSQL database tables, establishing a direct and automated connection between the data pipeline and the database.

6. Creating the SQL Database Table with Python

```python
with engine.connect() as conn:
    # Drop table if already exists (For Clean rebuild)
    conn.execute(text("DROP TABLE IF EXISTS scenario_table"))

# Create table DDL (run once).
ddl_statements = [
    """
    CREATE TABLE IF NOT EXISTS scenario_table (
    order_date DATE,
    category VARCHAR (50),
    sub_category VARCHAR (100),
    region VARCHAR (50),
    segment VARCHAR (50),
    sales NUMERIC,
    profit NUMERIC,
    discount NUMERIC
    );
    """
]
with engine.begin() as conn:
    for s in ddl_statements:
        conn.exec_driver_sql(s)
print("DDL executed (Table created).✅")
```

```
DDL executed (Table created).✅
```

Python + SQL Automation Pipeline

This project uses Python to automate database operations and execute SQL queries directly in PostgreSQL.

Database Connection

Established connection to PostgreSQL using SQLAlchemy engine

Table Creation

Programmatically created database tables from cleaned DataFrames

Data Insertion

Executed SQL queries for data insertion and transformation

Scenario Table Upload

Uploaded scenario table using to_sql() with replace mode

7. Uploading Scenario Table into PostgreSQL

```
import pandas as pd
scenario_table.to_sql('scenario_table', engine, if_exists = 'replace', index=False)
print("Scenario Table Uploaded Successfully")
scenario_table.head(8)
```

```
Scenario Table Uploaded Successfully
```

	order_date	sales	profit	discount	category	sub_category	region	segment
0	2016-11-08	261.9600	41.9136	0.00	Furniture	Bookcases	South	Consumer
1	2016-11-08	731.9400	219.5820	0.00	Furniture	Chairs	South	Consumer
2	2016-06-12	14.6200	6.8714	0.00	Office Supplies	Labels	West	Corporate
3	2015-10-11	957.5775	-383.0310	0.45	Furniture	Tables	South	Consumer
4	2015-10-11	22.3680	2.5164	0.20	Office Supplies	Storage	South	Consumer
5	2014-06-09	48.8600	14.1694	0.00	Furniture	Furnishings	West	Consumer
6	2014-06-09	7.2800	1.9656	0.00	Office Supplies	Art	West	Consumer
7	2014-06-09	907.1520	90.7152	0.20	Technology	Phones	West	Consumer

6. SQL Data Validation

Validated Data Before Loading into Power BI

Before connecting the dataset to Power BI, SQL queries were executed to validate the integrity, structure, and completeness of the data stored in PostgreSQL.

sql
SELECT * FROM scenario_table;

```
1  SELECT * FROM scenario_table
```

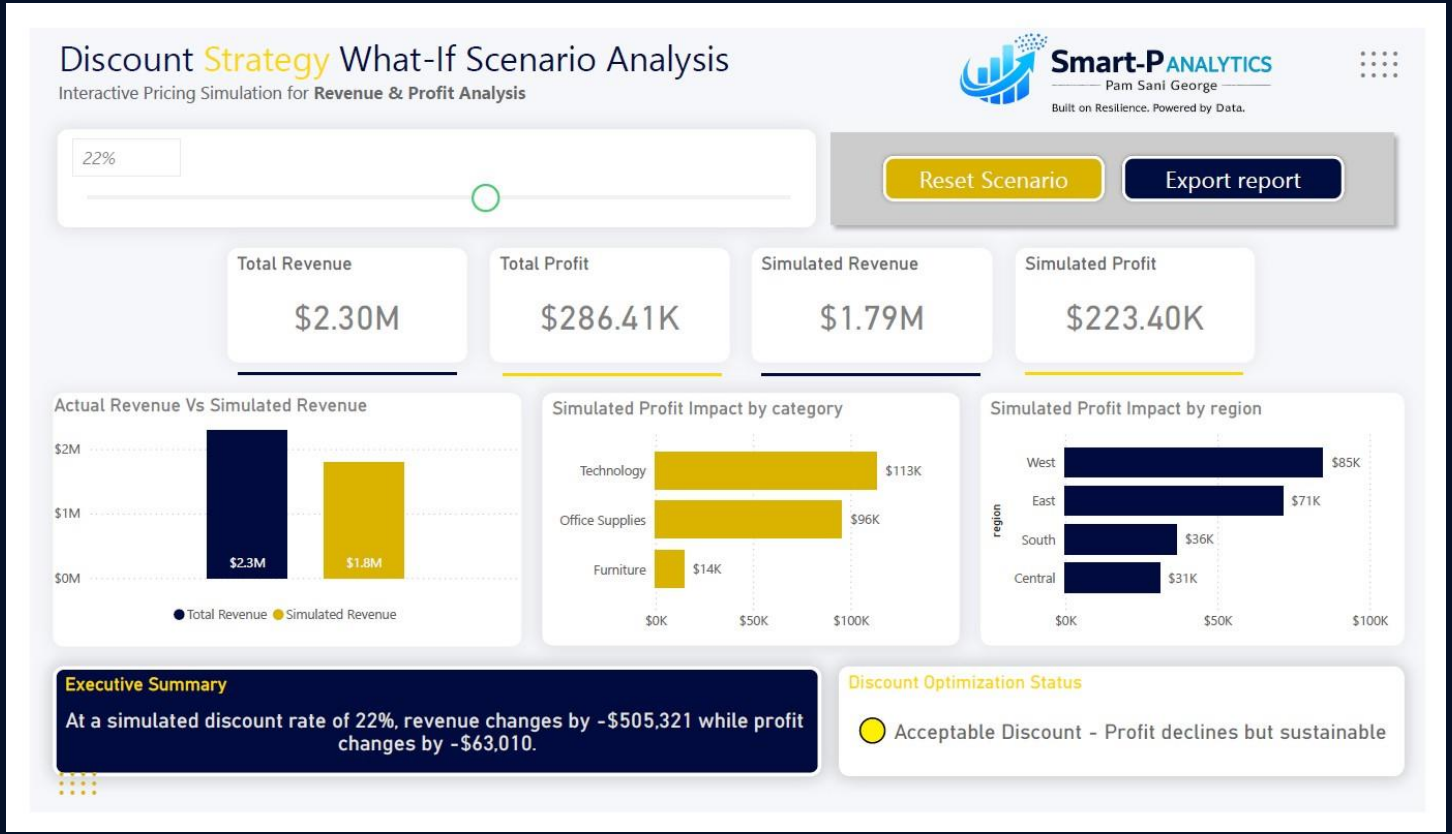
Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1

	order_date timestamp without time zone	sales double precision	profit double precision	discount double precision	category text	sub_category text	region text	segment text
1	2016-11-08 00:00:00	261.96	41.9136	0	Furniture	Bookcases	South	Consumer
2	2016-11-08 00:00:00	731.94	219.582	0	Furniture	Chairs	South	Consumer
3	2016-06-12 00:00:00	14.62	6.8714	0	Office Suppli...	Labels	West	Corporate
4	2015-10-11 00:00:00	957.5775	-383.031	0.45	Furniture	Tables	South	Consumer
5	2015-10-11 00:00:00	22.368	2.5164	0.2	Office Suppli...	Storage	South	Consumer
6	2014-06-09 00:00:00	48.86	14.1694	0	Furniture	Furnishings	West	Consumer
7	2014-06-09 00:00:00	7.28	1.9656	0	Office Suppli...	Art	West	Consumer
8	2014-06-09 00:00:00	907.152	90.7152	0.2	Technology	Phones	West	Consumer
9	2014-06-09 00:00:00	18.504	5.7825	0.2	Office Suppli...	Binders	West	Consumer
10	2014-06-09 00:00:00	114.9	34.47	0	Office Suppli...	Appliances	West	Consumer
11	2014-06-09 00:00:00	1706.184	85.3092	0.2	Furniture	Tables	West	Consumer
12	2014-06-09 00:00:00	911.424	68.3568	0.2	Technology	Phones	West	Consumer
13	2017-04-15 00:00:00	15.552	5.4432	0.2	Office Suppli...	Paper	South	Consumer
14	2016-12-05 00:00:00	407.976	132.5922	0.2	Office Suppli...	Binders	West	Consumer
15	2015-11-22 00:00:00	68.81	-123.858	0.8	Office Suppli...	Appliances	Central	Home Offi...
16	2015-11-22 00:00:00	2.544	-3.816	0.8	Office Suppli...	Binders	Central	Home Offi...
17	2014-11-11 00:00:00	665.88	13.3176	0	Office Suppli...	Storage	Central	Consumer
18	2014-05-13 00:00:00	55.5	9.99	0	Office Suppli...	Storage	West	Consumer

SQL Pipelines Executed Programmatically

SQL pipelines were executed programmatically using Python for full database automation, enabling a seamless and reproducible data workflow from raw input to Power BI-ready output.



7. Power BI Dashboard

Interactive What-If Scenario Analysis

The Power BI dashboard delivers a dynamic what-if scenario analysis experience, enabling users to simulate discount strategies and instantly evaluate their impact on revenue and profitability.

Python-Based Data Cleaning and Transformation

Raw dataset prepared and structured in Jupyter Notebook

PostgreSQL Database Integration

Cleaned data stored and managed in a relational database

SQL-Based Data Querying

Validation and transformation queries executed directly in PostgreSQL

Interactive Power BI Dashboard

Executive-grade visual reporting with dynamic slicers

What-If Discount Simulation

Real-time revenue and profit impact modeling

Profit Optimization Insights

Scenario-driven recommendations for pricing strategy

Business Value

This project demonstrates a full data pipeline from raw data to actionable business insights, making it highly valuable for organizations looking to implement data-driven decision making.

8. About the Analyst

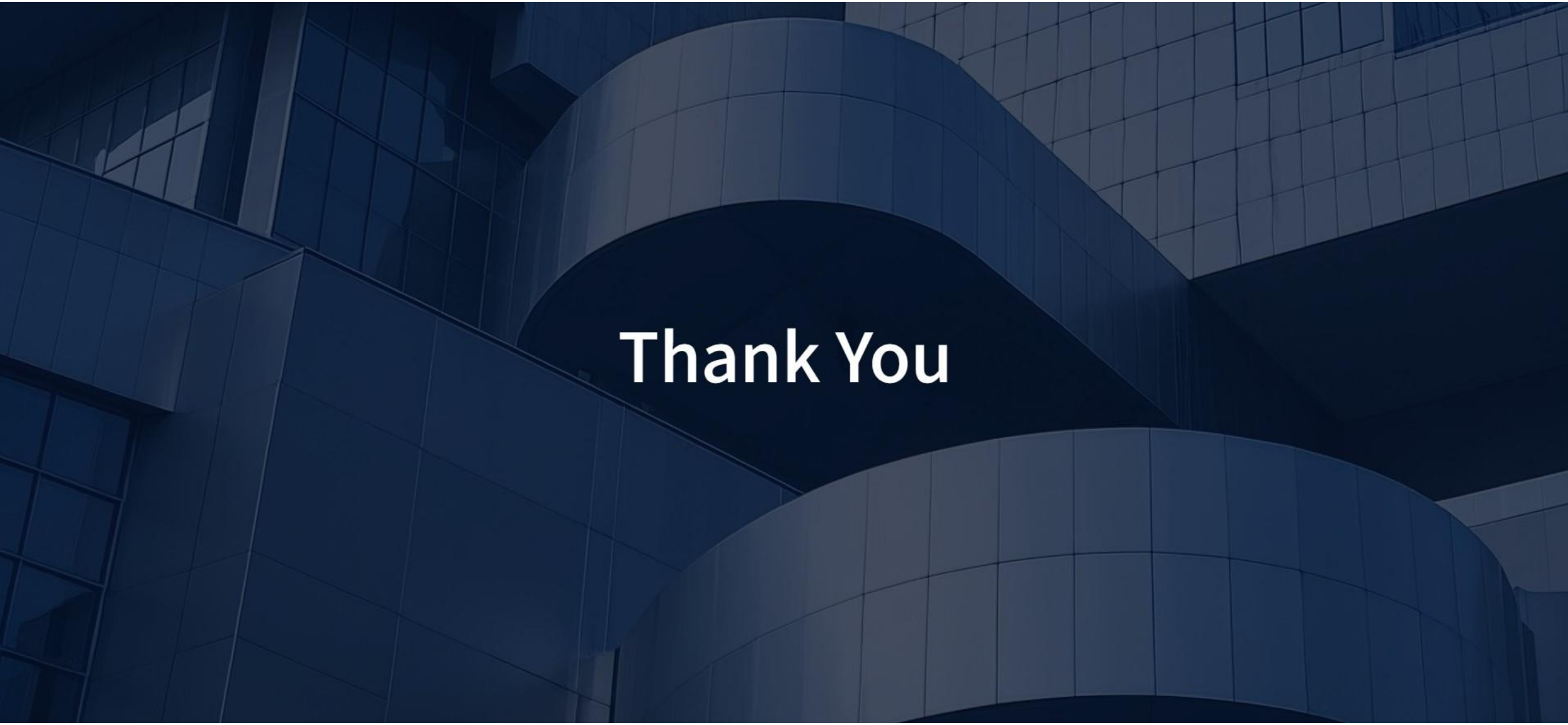
Pam Sani George

Role: Data Analyst | Business Intelligence Developer

Brand: Smart-P Analytics

Slogan: Built on Resilience. Powered by Data

LinkedIn: linkedin.com/in/pamsani



Thank You